SIMULATION OF POSITRON EMISSION TOMOGRAPHY (PET) IN GEANT4 AND MATLAB

Presented by:

Sakriya Dahal (32028)

Principles of Biomedical Engineering

INTRODUCTION

Positron Emission Tomography(PET) scan is an imaging test that reveals metabolic/biochemical functions using radiotracers emitting positrons.

Positrons collide with electrons, generating annihilation photons, captured to create 3D images.

Simulations optimize PET design, test algorithms, and analyze new systems securely and cost-effectively.

Geant4 is a robust toolkit which simulates particle interactions with matter for PET system analysis.

MATLAB facilitates data processing, analysis, and image reconstruction.

Integration of GEANT4 and MATLAB enhances PET system exploration and advances healthcare applications.

OBJECTIVES

1. To simulate the physical processes related to positron emission, the creation of annihilation photons, and their detection in a virtual PET scanner environment.
2. To evaluate system performance and enhance image quality by processing simulated data and reconstructing PET images using complex algorithms.

SIGNIFICANCE

1. Advancement in PET imaging technology.
2. Cost-effective system development.
3. Supports medical research.

Simulation of PET Scanner in Geant4

Geant4 successfully used for radioactive decay and PET setup.

Managed via the G4RadioactiveDecayPhysics Class in the physics list.

Accurate tracking of positrons and gamma rays from decay.

Decay Properties: Defined half-lives, decay modes, and isotopes for realistic behavior.

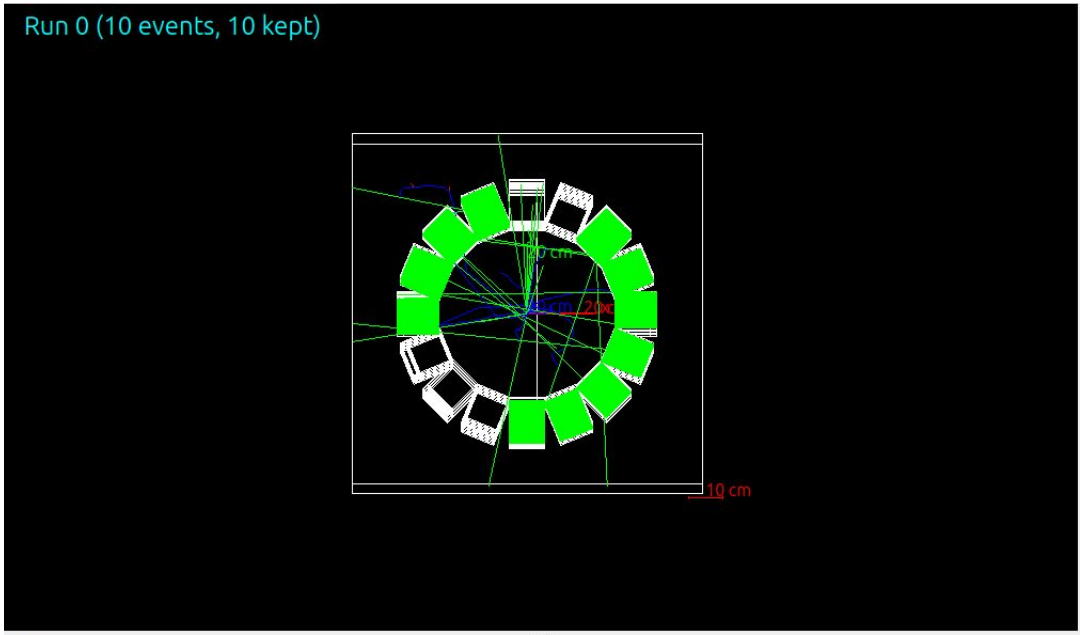


Figure: PET Scanner cross section

Simulation of PET Scanner in Geant4

Geometry and detector construction designed for accurate simulation.

Used G4VUserDetectorConstruction Class for realistic detector volumes (scintillators, photon detectors, shielding).

Geometry optimized to simulate positron interactions and energy deposition.

```
void MyDetectorConstruction::DefineMaterials()
{
    G4NistManager *nist = G4NistManager::Instance();

    worldMat = nist->FindOrBuildMaterial("G4_AIR");

    G4double energy[2] = {1.239841939*eV/0.9, 1.239841939*eV/0.2};

    G4double rindexWorld[2] = {1.0, 1.0};
    G4double rindexNaI[2] = {1.78, 1.78};
    G4double reflectivity[2] = {1.0, 1.0};

    G4MaterialPropertiesTable *mptWorld = new G4MaterialPropertiesTable();
    mptWorld->AddProperty("RINDEX", energy, rindexWorld, 2);

    Na = nist->FindOrBuildElement("Na");
    I = nist->FindOrBuildElement("I");
    NaI = new G4Material("NaI", 3.67*g/cm3, 2);
    NaI->AddElement(Na, 1);
    NaI->AddElement(I, 1);

    G4double fraction[2] = {1.0, 1.0};

    G4MaterialPropertiesTable *mptNaI = new G4MaterialPropertiesTable();
    mptNaI->AddProperty("RINDEX", energy, rindexNaI, 2);
    mptNaI->AddProperty("SCINTILLATIONCOMPONENT1", energy, fraction, 2);
    mptNaI->AddConstProperty("SCINTILLATIONYIELD",38./keV);
    mptNaI->AddConstProperty("RESOLUTIONSCALE",1.0);
    mptNaI->AddConstProperty("SCINTILLATIONTIMECONSTANT1", 250.*ns);
    mptNaI->AddConstProperty("SCINTILLATIONYIELD1",1.);

    NaI->SetMaterialPropertiesTable(mptNaI);

    worldMat->SetMaterialPropertiesTable(mptWorld);

    mirrorSurface = new G4OpticalSurface("mirrorSurface");

    mirrorSurface -> SetType(dielectric_metal);
    mirrorSurface -> SetFinish(ground);
    mirrorSurface -> SetModel(unified);

    G4MaterialPropertiesTable *mptMirror = new G4MaterialPropertiesTable();

    mptMirror -> AddProperty("REFLECTIVITY", energy, reflectivity, 2);

    mirrorSurface -> SetMaterialPropertiesTable(mptMirror);
}
```

Simulation of PET Scanner in Geant4

A wide set up of scintillators was constructed out of NaI blocks using material classes in Geant4.

The properties of these scintillators were set up to reflect photons that can be detected and used for imaging.

```
void MyDetectorConstruction::ConstructScintillator()
{
    solidScintillator = new G4Box("solidScintillator", 5*cm, 5*cm, 6*cm);

    logicScintillator = new G4LogicalVolume(solidScintillator, NaI, "logicalScintillator");

    G4LogicalSkinSurface *skin = new G4LogicalSkinSurface("skin", logicWorld, mirrorSurface);

    solidDetector = new G4Box("solidDetector", 1*cm, 5*cm, 6*cm);

    logicDetector = new G4LogicalVolume(solidDetector, worldMat, "logicDetector");

    fScoringVolume = logicScintillator;

    for(G4int i = 0; i<6; i++)
    {
        for(G4int j = 0; j<16; j++)
        {
            G4Rotate3D rotZ(j*22.5*deg, G4ThreeVector(0,0,1));
            G4Translate3D transXScint(G4ThreeVector(5./tan(22.5/2*deg)*cm + 5.*cm, 0*cm, -40*cm + i*15*cm));

            G4Translate3D transXDet(G4ThreeVector(5./tan(22.5/2*deg)*cm + 6.*cm, 0*cm, -40*cm + i*15*cm));

            G4Transform3D transformScint = (rotZ)*(transXScint);
            G4Transform3D transformDet = (rotZ)*(transXDet);

            physScintillator = new G4PVPlacement(transformScint, logicScintillator,"physScintillator",logicWorld, false,0, true);

            physDetector = new G4PVPlacement(transformDet, logicDetector,"physDetector",logicWorld, false,0, true);
        }
    }
}
```

Simulation of PET Scanner in Geant4

Photon detectors were also generated from the same class in Geant4.

Detectors would detect the position and momentum of the photons generated by the scintillators, and their wavelengths would be calculated.

Trajectory of the photons with desired wavelength would be recorded.

```
G4bool MySensitiveDetector::ProcessHits(G4Step *aStep, G4TouchableHistory *ROhist)
{
    G4Track *track = aStep -> GetTrack();

    track -> SetTrackStatus(fStopAndKill);

    G4StepPoint *preStepPoint = aStep -> GetPreStepPoint();
    G4StepPoint *postStepPoint = aStep -> GetPostStepPoint();

    G4ThreeVector posPhoton = preStepPoint -> GetPosition();
    G4ThreeVector momPhoton = preStepPoint -> GetMomentum();

    G4double wlen = (1.239841939 *eV/momPhoton.mag())*1E+03; //E = h miu
    //G4cout << "Photon position: " << posPhoton << G4endl;

    const G4VTouchable *touchable = aStep -> GetPreStepPoint() -> GetTouchable();

    G4int copyNo =  touchable -> GetCopyNumber();
    //G4cout << "Copy number: " << copyNo << G4endl;

    G4VPhysicalVolume *physVol = touchable -> GetVolume();
    G4ThreeVector posDetector = physVol -> GetTranslation();
    #ifndef G4MULTITHREADED
        //G4cout << "Detector position: " << posDetector << G4endl;
        //G4cout << "Photon Wavelength: " << wlen << G4endl;
    #endif

    G4int evt = G4RunManager::GetRunManager()->GetCurrentEvent()->GetEventID();

    G4AnalysisManager *man = G4AnalysisManager::Instance();

    man -> FillNtupleIColumn(0, 0, evt);
    man -> FillNtupleDColumn(0, 1, posPhoton[0]);
    man -> FillNtupleDColumn(0, 2, posPhoton[1]);
    man -> FillNtupleDColumn(0, 3, posPhoton[2]);
    man -> FillNtupleDColumn(0, 4, wlen);
    man -> AddNtupleRow(0);


    if(G4UniformRand() < quEff -> Value(wlen)){
    man -> FillNtupleIColumn(1, 0, evt);
    man -> FillNtupleDColumn(1, 1, posDetector[0]);
    man -> FillNtupleDColumn(1, 2, posDetector[1]);
    man -> FillNtupleDColumn(1, 3, posDetector[2]);
    man -> AddNtupleRow(1);
    }

    return true;
}
```

Simulation of PET Scanner in Geant4

Source of Fluorine-18 was also generated within Geant4 to simulate a simple arbitrary emission that imitates emissions from a human body.

```
MyPrimaryGenerator::~MyPrimaryGenerator()
{
    delete fParticleGun;
}

void MyPrimaryGenerator::GeneratePrimaries(G4Event *anEvent)
{
    G4ParticleDefinition *particle = fParticleGun -> GetParticleDefinition();

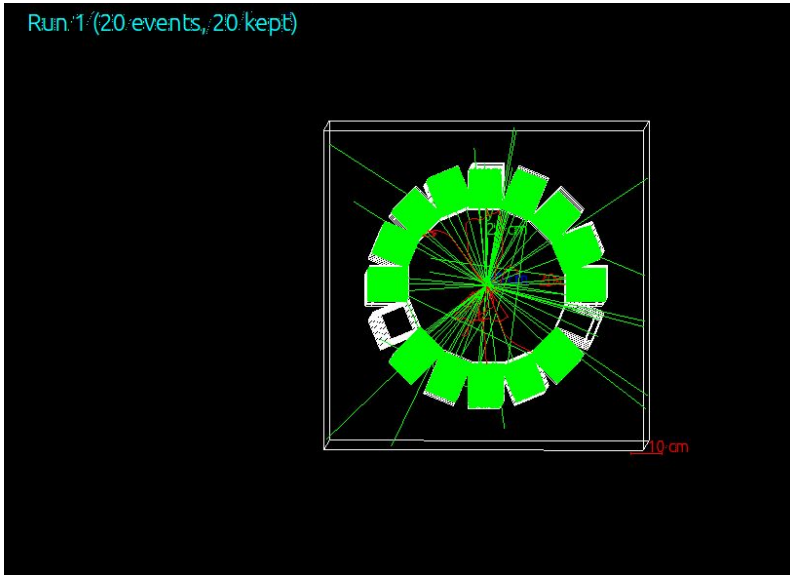
    if(particle == G4Geantino::Geantino())
    {
        G4int Z = 9;
        G4int A = 18;

        G4double charge = 0. *eplus;
        G4double energy = 0. *keV;

        G4ParticleDefinition *ion = G4IonTable::GetIonTable() -> GetIon(Z,A,energy);

        fParticleGun -> SetParticleDefinition(ion);
        fParticleGun -> SetParticleCharge(charge);
  }


    fParticleGun -> GeneratePrimaryVertex(anEvent);
}
```



Simulation of PET Scanner in Geant4

The F-18 undergoes beta-plus decay and emits a positron.

This positron has a short lifespan until it annihilates with an electron.

The annihilation produces two gamma photons, each with 511keV energy, in opposite direction.



Figure: emission of gamma photons through F-18 annihilation

Simulation of PET Scanner in Geant4

The NaI scintillators detect the gamma photons.

The scintillator blocks act as mirrors and convert the gamma photons into visible light.

They are detected by detectors that usually consist of PhotoMultiplier Tubes.

This data is then used for 3D imaging.

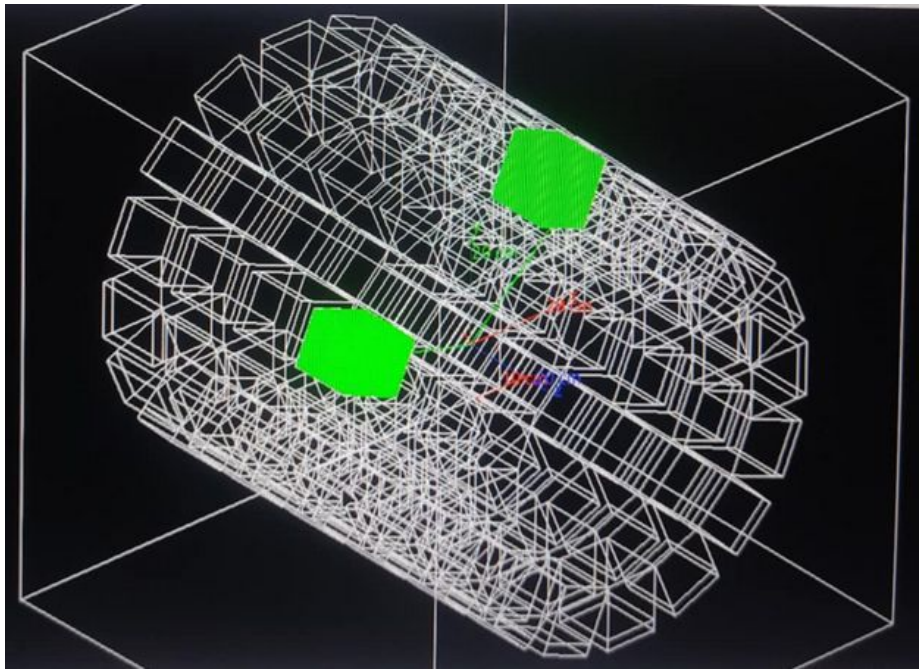
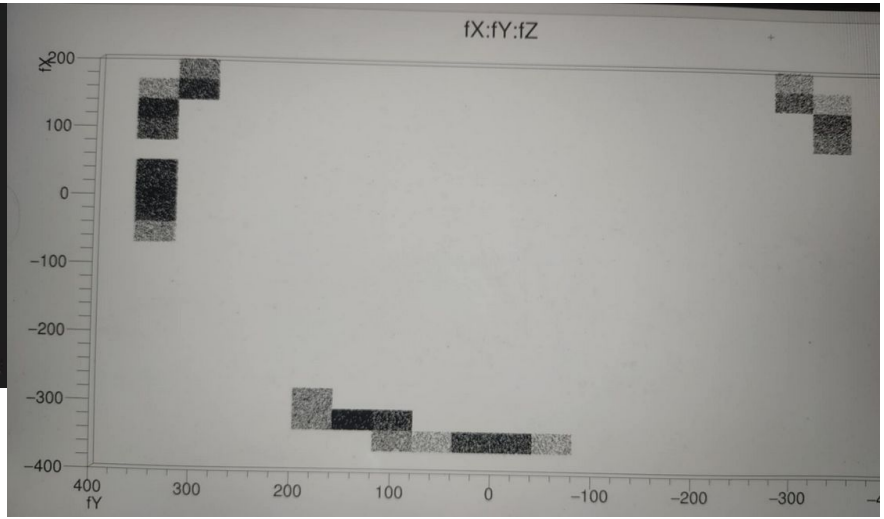


Figure: emission of gamma photons through F-18 annihilation

Simulation of PET Scanner in Geant4

Energy deposition data collected in the detectors.

```
G4WT0 > Energy deposition: 0.334843
G4WT0 > Energy deposition: 0
G4WT1 > Energy deposition: 1.11249
G4WT0 > Energy deposition: 1.022
G4WT1 > Energy deposition: 0.510999
G4WT0 > Energy deposition: 0.769901
G4WT1 > Energy deposition: 0.891349
G4WT1 > Energy deposition: 0.134643
G4WT0 > Energy deposition: 0.733683
G4WT0 > Energy deposition: 0.708973
10 events have been kept for refreshing and/or reviewing.
```



Data Acquisition in PET Imaging

PET employs a radiotracer (such as Fluorodeoxyglucose, or FDG) that is injected into the body.

This emits positrons which collide with electrons resulting in annihilation and the emission of back-to-back photons.

These photons move along different directions to the PET scanner detectors, which captures the occurrences of photon emission.

Detectors in the PET scanner capture these photons along lines of response.

Generating Photon Pair Lines

A random photon source is chosen inside the phantom for every iteration.

Random x and y coordinates along with a random angle is assigned for the emission of the photons.

The path linking the photon source to its target is established through Bresenham's Line Algorithm (digital algorithm used to draw straight lines between two points).

This ensures that the photon path is accurately simulated, corresponding to actual PET imaging.

Simulated photon paths are essential for generating accurate sinogram data for reconstruction.

Sinograms

It is a 2D representation of projection data from various angles.

It is essential for the reconstruction of tomographic images (PET,CT).

This displays attenuation information for every angle.

It is based on Radon transform(projection function).

This helps to display the sinogram image generated by the code.

Structure of a Sinogram

The sinogram fundamentally serves as a matrix where:

The horizontal axis depicts various angles (perspectives) from which the data is collected.

The vertical axis represents positions along the LORs.

Every pixel in the sinogram indicates a measurement of the overall radioactivity (or photon counts) observed along a particular LOR at a certain angle.

Thus, a sinogram consists of rows(indicating various angles) and columns (indicating positions along LORs).

Perpendicular Lines in Sinogram

While the scanner rotates, it collects data along perpendicular lines (LORs) within the object.

Every line indicates the overall photon count along a given trajectory within the object.

These lines constitute the foundation of the sinogram obtaining data from different angles for precise reconstruction.

Radon Transform in PET

The Radon Transform represents the true object’s projections from the different angles creating sinograms.

It shows how a line integral (sum of activities) is derived along a specific line of response (s, phi).

This is essential for both back projection and iterative reconstruction.

Back Projection

It is a method to recreate the original image by back projecting the sinogram data.

Each row of sinogram is projected back into the image domain which helps to reconstruct the image.

This is a simple and initial approach to image reconstruction.

It leads to unclear images when used by itself, but helps as an initial guess in iterative techniques.

This displays the back-projected image resulting from the iterative reconstruction steps.

Reconstruction Using Sinogram Data

The sinogram data is important for iteratively reconstructing the image.

The Algebraic Reconstruction Technique (ART) is utilized in MATLAB for enhancement.

ART facilitates high resolution reconstruction by progressively enhancing the image through several iterations.

Code Implementation

Simulates the forward projection of a phantom object using the Radon Transform and reconstructs the image iteratively.

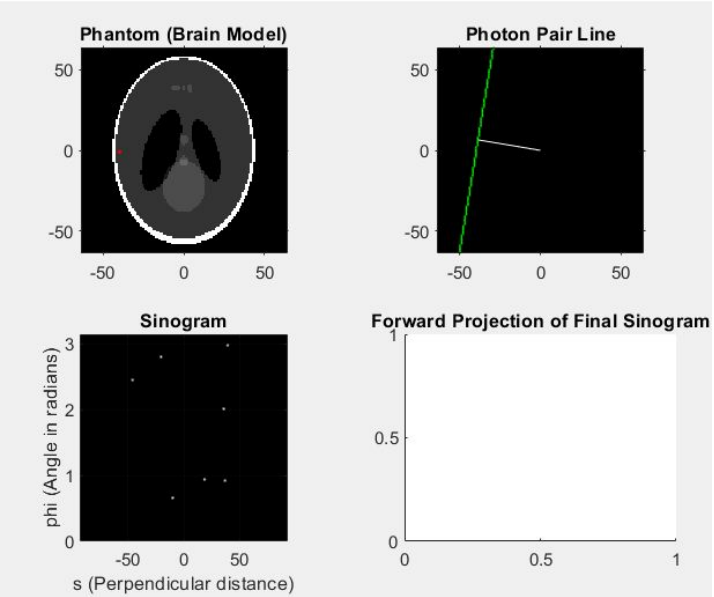


Figure: Photon Pair Generation, Sinogram Formation, and Forward Projection at 7th Iteration in PET Imaging

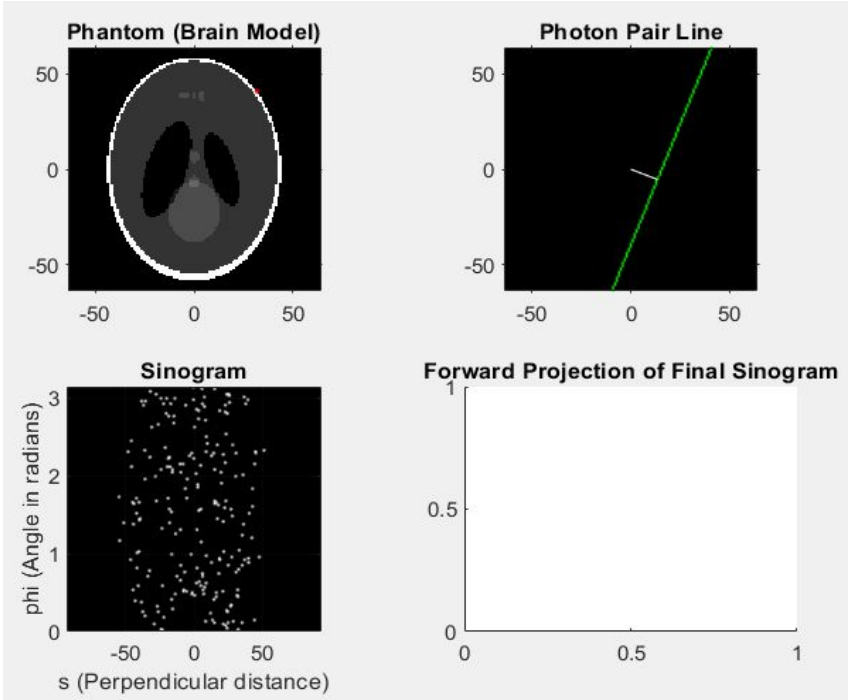


Figure: Photon Pair Generation, Sinogram Formation, and Forward Projection at 300th Iteration in PET Imaging

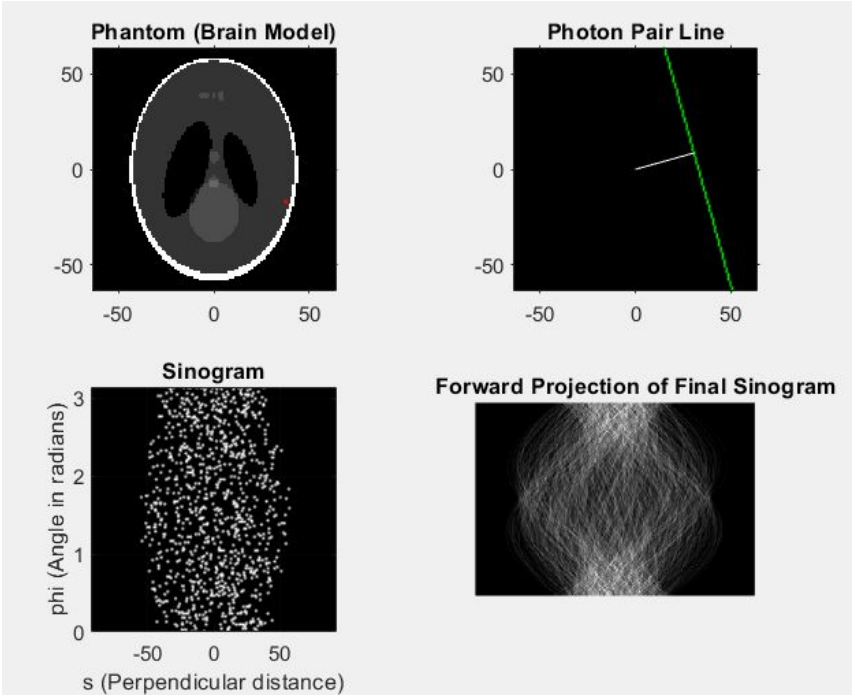


Figure: Photon Pair Generation, Sinogram Formation, and Forward Projection at 1000th Iteration in PET Imaging

Phantom- Based Reconstruction

The forward projection of final sinogram demands the significant computational resources i.e. requires a infinite no. of lines (ideal case).

This is time-consuming.

To address the limitations of time and computational resources, a phantom is used to stimulate the process and perform the reconstruction without depending on the real data.

The phantom offers a simplified model for assessing the algorithm and confirm the reconstruction technique before applying it to real-world sinograms.

Implements back projection using measured data to approximate the original object.

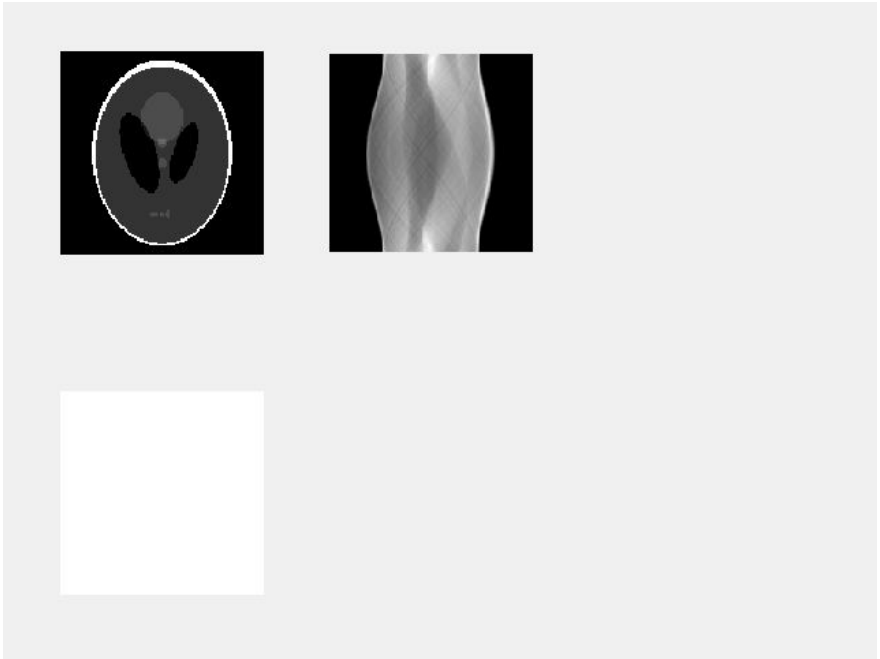


Figure: Iterative Reconstruction of PET Image using Ration-Based Update and Back- Projection

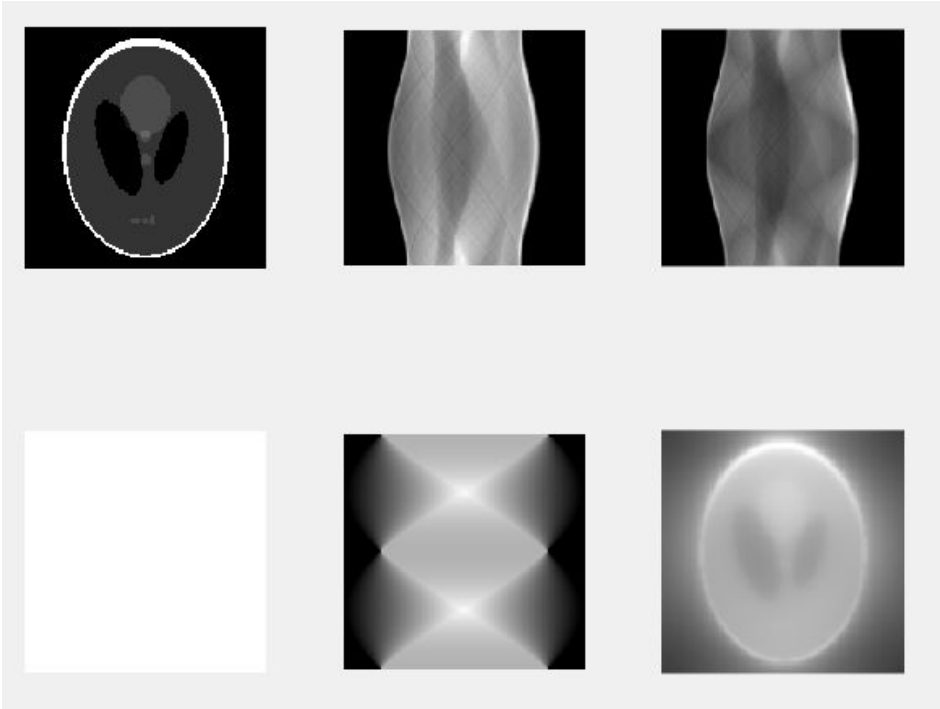
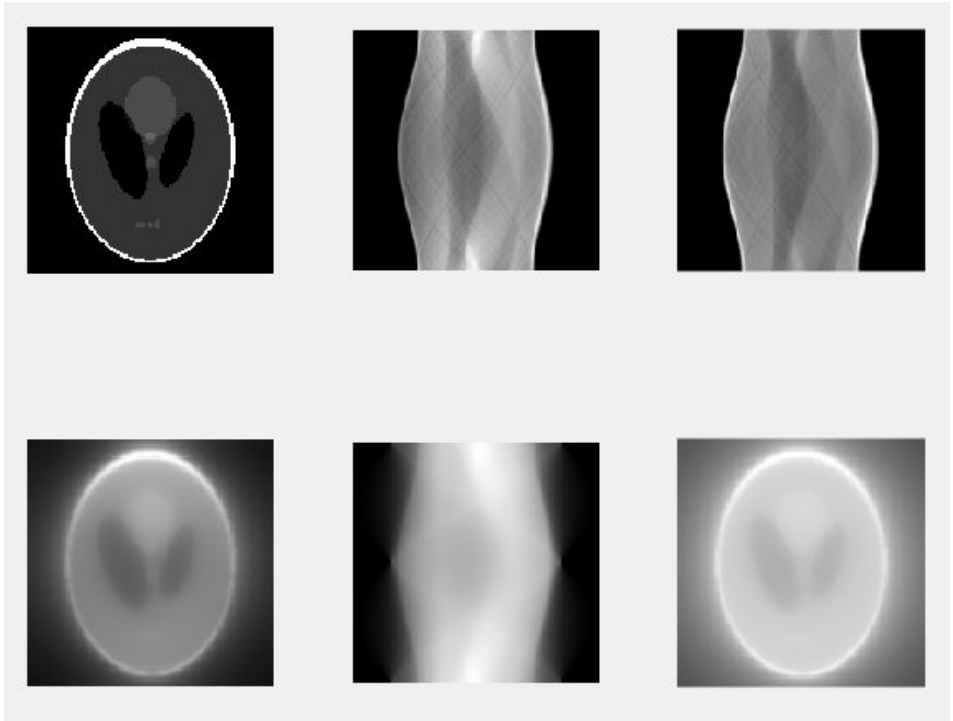
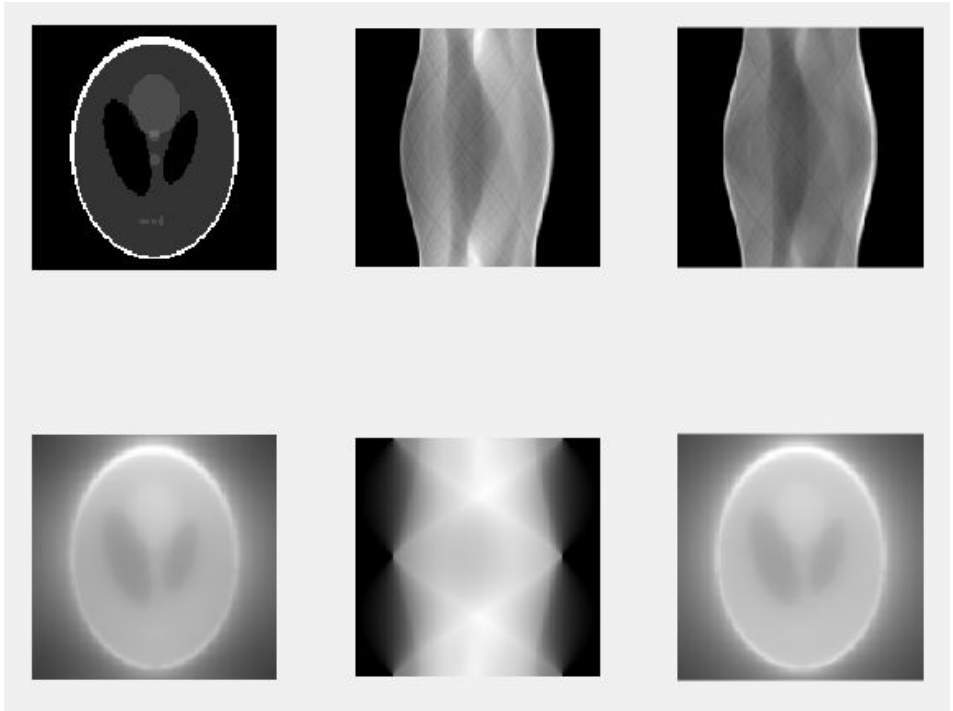


Figure: True phantom, Measured sinogram, Ration between measured & forward projected data, Initial reconstructed image, Forward Projection of reconstructed image and Back-projected correction ratio





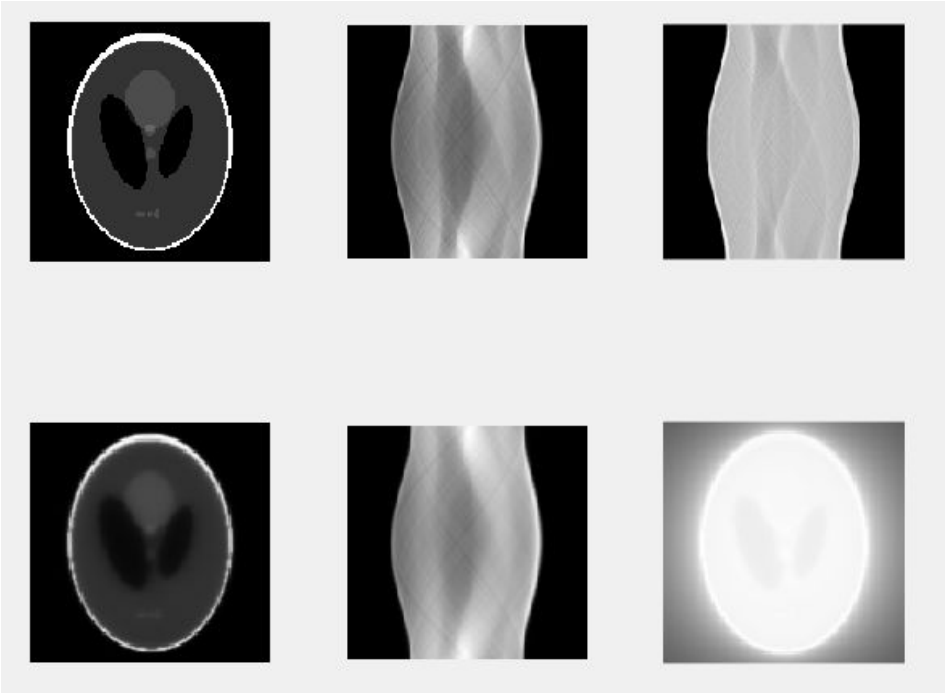


Figure: True phantom, Measured sinogram, Ration between measured & forward projected data, Initial reconstructed image, Forward Projection of reconstructed image and Back-projected correction ratio

Conclusion

Geant4 offers an accurate simulation of radiation interactions and the generation of the photon pairs.

MATLAB provides fast calculations and effective management of large datasets.

The integration of Geant4 ad MATLAB allows the complete simulation of photon pair generation, detection and image reconstruction.

References

[1] Schmidt, (2020) geant4-tutorial [Source code].
https://github.com/MustafaSchmidt/geant4-tutorial

[2] A. J. Reader, Principles of Iterative Reconstruction for Emission Tomography, in Handbook of Nuclear Medicine and Molecular Imaging for Physicists, P. S. Chua and T. K. Lee, Eds. Boca Raton, FL: CRC Press, 2011.

Appendix

```
clear; close all;

xd = 128;
num_iterations = 1000;
num_angles = 180;
angles = linspace(0, 180, num_angles);
azi_angles = 1:1:180; % Define globally

true_object = abs(phantom(xd));

figure;

subplot(2, 2, 1);
imshow(true_object, [0 max(true_object(:))], 'XData', [-64 64], 'YData', [-64 64]);
axis on;
axis([-64 64 -64 64]);
set(gca, 'YDir', 'normal');
title('Phantom (Brain Model)');
hold on;

subplot(2, 2, 2);
imshow(zeros(xd), 'XData', [-64 64], 'YData', [-64 64]);
axis on;
axis([-64 64 -64 64]);
set(gca, 'YDir', 'normal');
title('Photon Pair Line');
hold on;
```

```
subplot(2, 2, 3);
sinogram = zeros(180, 185);
imagesc(linspace(-92.5, 92.5, 185), linspace(0, pi, 180), sinogram);
colormap([1 1 1; 0 0 0]);
axis on;
axis([-92.5 92.5 0 pi]);
set(gca, 'YDir', 'normal');
xlabel('s (Perpendicular distance)');
ylabel('phi (Angle in radians)');
title('Sinogram');
grid on;
hold on;
pbaspect([1 1 1]);

subplot(2, 2, 4);
title('Forward Projection of Final Sinogram');
hold on;

for it = 1:num_iterations
    while true
        x = randi([1, xd]);
        y = randi([1, xd]);
        if true_object(y, x) > 0
            break;
        end
    end

    x_centered = x - xd / 2;
    y_centered = y - xd / 2;

    angle_idx = randi([1, num_angles]);
    theta = angles(angle_idx);

    x2 = round(x_centered + 2 * xd * cosd(theta));
    y2 = round(y_centered + 2 * xd * sind(theta));
    x3 = round(x_centered - 2 * xd * cosd(theta));
    y3 = round(y_centered - 2 * xd * sind(theta));
```

```
[x_line1, y_line1] = bresenham_line(x_centered, y_centered, x2, y2);
[x_line2, y_line2] = bresenham_line(x_centered, y_centered, x3, y3);
x_line = [x_line1; x_line2];
y_line = [y_line1; y_line2];
valid_idx = x_line >= -64 & x_line <= 64 & y_line >= -64 & y_line <= 64;
x_line = x_line(valid_idx);
y_line = y_line(valid_idx);

A = sind(theta);
B = -cosd(theta);
C = -(A * x_centered + B * y_centered);

s_unsigned = abs(C) / sqrt(A^2 + B^2);

rho_x = -A * C / (A^2 + B^2);
rho_y = -B * C / (A^2 + B^2);

phi = mod(atan2d(rho_y, rho_x), 180);

if rho_y >= 0
    s = s_unsigned;
else
    s = -s_unsigned;
end

subplot(2, 2, 1);
imshow(true_object, [0 max(true_object(:))], 'XData', [-64 64], 'YData', [-64 64])
axis on;
axis([-64 64 -64 64]);
set(gca, 'YDir', 'normal');
title('Phantom (Brain Model)');
hold on;
plot(x_centered, y_centered,'r.', 'MarkerSize', 5);
hold off;

    subplot(2, 2, 2);
    imshow(zeros(xd), 'XData', [-64 64], 'YData', [-64 64]);
    axis on;
    axis([-64 64 -64 64]);
    set(gca, 'YDir', 'normal');
    title('Photon Pair Line');
    hold on;
    plot(x_line, y_line, 'g.', 'MarkerSize', 2);
    plot([0 rho_x], [0 rho_y], 'w-', 'LineWidth', 0.5);
    hold off;

    subplot(2, 2, 3);
    plot(s, deg2rad(phi), 'w.', 'MarkerSize', 3);
    drawnow;

    sinogram(angle_idx, :) = sinogram(angle_idx, :) + histcounts(s, linspace(-92.5, 92.5, 186));
end

% Forward Projection of Final Sinogram
fp = radon(sinogram, azi_angles);
subplot(2, 2, 4);
imshow(fp', [0 max(fp(:))]);
title('Forward Projection of Final Sinogram');
drawnow;
```

```
function [x, y] = bresenham_line(x1, y1, x2, y2)
    x = [];
    y = [];
    dx = abs(x2 - x1);
    dy = abs(y2 - y1);
    sx = sign(x2 - x1);
    sy = sign(y2 - y1);

    if dy <= dx
        err = dx / 2;
        while x1 ~= x2
            x = [x; x1];
            y = [y; y1];
            err = err - dy;
            if err < 0
                y1 = y1 + sy;
                err = err + dx;
            end
            x1 = x1 + sx;
        end
    else
        err = dy / 2;
        while y1 ~= y2
            x = [x; x1];
            y = [y; y1];
            err = err - dx;
            if err < 0
                x1 = x1 + sx;
                err = err + dy;
            end
            y1 = y1 + sy;
        end
    end
    x = [x; x2];
    y = [y; y2];
end
```

```
clear; close all;

image_size = 128; % x and y dimension of phantom

figure(1);
phantom_image = abs(phantom(image_size));

subplot(2,3,1), imshow(phantom_image, [0 max(phantom_image(:))]);

angle_range = 1:1:180;

sinogram_data = radon(phantom_image);

subplot(2,3,2), imshow(sinogram_data', [0 max(sinogram_data(:))]);

reconstructed_image = ones(size(phantom_image)); % initial reconstructed image

sinogram_ones = ones(size(sinogram_data));

sensitivity_map = iradon(sinogram_ones, angle_range, 'none', image_size);

subplot(2,3,4), imshow(reconstructed_image, [0 max(reconstructed_image(:))]);
pause

for iteration = 1:20

    forward_projection = radon(reconstructed_image, angle_range);
    subplot(2,3,5), imshow(forward_projection', [0 max(forward_projection(:))]);

    projection_ratio = sinogram_data ./ (forward_projection + 0.0001);
    subplot(2,3,3), imshow(projection_ratio', [0 max(projection_ratio(:))]);
    backprojected_ratio = iradon(projection_ratio, angle_range, 'none', image_size);

    subplot(2,3,6), imshow(backprojected_ratio, [0 max(backprojected_ratio(:))]);

    reconstructed_image = reconstructed_image .* backprojected_ratio ./ sensitivity_map;

    pause

    subplot(2,3,4), imshow(reconstructed_image, [0 max(reconstructed_image(:))]);
end
```

THANK YOU

Any Queries?